

A new data reduction approach over the stream processor architecture

Qingkui Chen, Li Xiao, Songlin Zhuang*

*School of Optical-Electrical and Computer Engineering
University of Shanghai for Science and Technology, Shanghai, P.R.China
Email: chenqingkui@gmail.com

Abstract—In order to solve the parallel processing problem of mass data, stream processor develops quickly in recent years. The essence of stream processor is getting high performance according to executing thousands of threads parallelly. It is widely used in scientific computing, data mining, video quality analysis and other fields. But mass threads means the high costs of thread synchronization. Mass computing time is wasted in the reduction of thread data. This paper provides a binary reduction algorithm based on three basic reduction algorithms according to CUDA parallel computing model. Experiments show that the performance of binary reduction algorithm is better than three basic reduction algorithms and it has great scalability.

I. INTRODUCTION

With the rapid development of Internet and Information technology, the processing of mass information becomes more and more important. Mass data have the character of large-scale concurrency. Multicore CPU and stream processor[1] are designed to solve this kind of problems. The CUDA[2] developed by NVIDIA and the STREAM[3] developed by AMD laid a very important foundation for stream processors application in many fields such as scientific computing[4], data mining[5], decoding research[6], robust motion tracking [7]. The essence of stream processor is getting high performance according to executing thousands of threads parallelly. We use stream processor to analysis the quality of 3G video. The whole analysis includes blur effect analysis[8] and blocking effect analysis[9]. In the process of analysis, each image in 3G video is divided into thousands of data blocks and each data block is processed in a thread. This way can decrease the computing time of quality analysis. Since using a large number of threads will produce serious synchronization problem, the synchronization algorithm among the GPU threads becomes very important. The synchronization among the thousands of GPU threads involves the computing structure of stream processor, the multilevel cache structure and the scheduling way of threads. Paper[10], paper[11] and paper[12] discuss these factors but the discussions only focus on small-scale synchronization. This paper provides four reduction algorithms according to the CUDA computing model. Algorithm1, algorithm2 and algorithm3 are related to processor structure. Algorithm4 is binary reduction algorithm. It is not limited by processor structure and it supports threads synchronization among the thread blocks. The computing time of binary reduction algorithm is less than three basic reduction algorithms

and it has great scalability to solve synchronization problem of mass threads.

The rest of the paper is organized as follows: chapter 2 introduces parallel blur degree analysis architecture and chapter 3 introduces binary reduction algorithm. Experiment results are shown in chapter 4. Conclusions are summarized in chapter 5.

II. PARALLEL BLUR DEGREE ANALYSIS ARCHITECTURE

This chapter will introduce parallel blur degree analysis architecture. It is one of the application backgrounds of binary reduction algorithm. Section A introduces blur degree analysis method. Section B introduces blur degree analysis algorithm on CUDA. The thread data reduction problem in parallel blur degree analysis architecture will be discussed in section C.

A. Blur Degree Analysis Method

This section mainly discusses non-reference blur degree analysis method, which provides objective evaluation for different blur degree. The blur degree analysis method takes Y component of YUV video as input and output the blur degree as Q. The evaluation can be divided into 5 phases, which are shown as below:

1) Gradient computing

We use *canny* operator [13] to detect the edge of the image, with which we can get gradient image $Grad(x, y)$ and edge binarization image $edge(i, j)$.

2) Edge scanning

We scan the gradient image and compute the width information. Through line by line scanning image gradient and column by column scanning image gradient, we can get the width information of gradient edge.

Before the scanning process, the limit of edge width must be set greater than the one-tenth of image width in the corresponding direction. If the width information is greater than the limit, it will be recognized as the edge in the orthogonal direction and will be ignored.

Taking gradient image $Grad(x, y)$ as the input of step 2, it can produce the edge width in both horizontal and vertical direction, the quantity of convex peaks in both horizontal and vertical direction, and the gradient strength. The output procedure is shown as below:

For each row of the gradient image, it searches from the first nonzero element until meeting an element equal to zero.

The area between the first nonzero element and the element previous to the element equal to zero is called an edge. The distance between them is the edge width and the gradient strength of the edge is the greatest gradient value in the edge. The quantities of convex peak in the horizontal direction equal to the quantities of the edge in the horizontal direction. We can use similar steps to scan columns.

3) Edge width strength computing

According to Equation 1, given the width information of rows and columns, we can compute the edge width strength of gradient image

$$H = \frac{1}{N} \sum_{i=1}^N \frac{W_h^i * S}{V_h^i} + \frac{1}{M} \sum_{i=1}^M \frac{W_v^i * S}{V_v^i} \quad (1)$$

In Equation 1, W_h^i is the edge width in the horizontal direction while W_v^i is the edge width in the vertical direction, N and M represent the quantity of convex peaks in the horizontal direction and the vertical direction respectively, W_h^i and W_v^i represent the gradient strength in the horizontal and the vertical direction respectively, H is the edge width strength of gradient image and S is the max gradient value.

4) Total length of edge computing

We can compute total length of edge ($L_{h,v}$) from edge binary image. Total length of edge is the quantity of nonzero pixels in edge binary image.

5) Blur degree computing

From above, we can get the blur image quality evaluation model in Equation 2:

$$Q = \frac{2L_{h,v}}{H(M+N)} \quad (2)$$

The Q is the blur degree of image. The greater value of Q , the less blur effect appears, and vice versa.

B. Blur Degree Analysis Algorithm on CUDA

Blur degree analysis algorithm on CUDA includes one main function and four kernel functions. Main function runs in CPU and kernel functions run in GPU. They are shown as below:

a) Main function

Main function is used to prepare data. Firstly, it gets Y component of YUV images, size of YUV images and save them in the host memory. Secondly, it copies these data from host memory to graphics memory. Thirdly, it calls four kernel functions to compute blur degree of YUV images. Fourthly, it copies computing result from graphics memory to host memory.

b) GradientComputing functionKernel function

One thread is used to compute gradient and total length of edge in a $z1 * z2$ area of one image. Firstly, it finds the area which current thread deals with. Secondly, it uses *canny* operator to compute gradient and edge binarization result of $pixel(i, j)$. Thirdly, it checks the edge binarization result of $pixel(i, j)$. If the result doesn't equal to 0, the total length of edge adds 1. Repeat step two and step three until all the pixels in the area are processed and then we can get a gradient

image of current YUV image. Threads distribution is shown in Table1.

TABLE I
THREADS DISTRIBUTION IN GRADIENT COMPUTING FUNCTION

Parameter	Default Value
Grid	Number
Block	(M/z1, N/z2)

In Table 1, *Number* is the number of YUV images. M and N are the max width and height of image that can be supported. $z1 * z2$ is the size of area in YUV image which can be processed by one thread.

c) HorizontalScan functionKernel function

One thread is used to scan a row of one image in the horizontal direction. Firstly, it finds the row which current thread deals with. Secondly, it uses edge scanning method which is introduced in Section2 to scan the row. When all the threads which deal with one image are finished, we get the sum of $\frac{W_k^i * S}{V_k^i}$ and the value of N for the image. Threads distribution is shown in Table2.

TABLE II
THREADS DISTRIBUTION IN HORIZONTALSCAN FUNCTION

Parameter	Default Value
Grid	Number
Block	N

In Table 2, *Number* is the number of YUV images. N is the max height of image that can be supported.

d) VerticalScan functionKernel function

One thread is used to scan a column of one image in the vertical direction. Firstly, it finds the column which current thread deals with. Secondly, it uses edge scanning method which is introduced in Section2 to scan the column. When all the threads which deal with one image are finished, we get the sum of $\frac{W_v^i * S}{V_v^i}$ and the value of M for the image. Threads distribution is shown in Table3.

TABLE III
THREADS DISTRIBUTION IN VERTICALSCAN FUNCTION

Parameter	Default Value
Grid	Number
Block	M

In Table 3, *Number* is the number of YUV images. M is the max width of image that can be supported.

e) BlurDegreeCompute functionKernel function

One thread is used to compute blur degree of one image. Firstly, it computes the total edge length of one image according to the total edge lengths of all sub area. Then it uses Equation 2 to compute blur degree of the image. Threads distribution is shown in Table4.

TABLE IV
THREADS DISTRIBUTION IN VERTICALSCAN FUNCTION

Parameter	Default Value
Grid	1
Block	Number

In Table 4, *Number* is the number of YUV images.

C. Reduction Problem

In order to analyze the influence of thread distribution to the computing time of parallel blur degree analysis architecture, we perform two experiments. The first experiment is changing the number of threads in each thread block and keep the number of thread blocks unchanged. The experiment result is presented in Fig. 1.

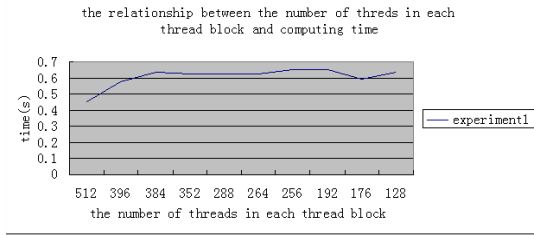


Fig. 1. binary reduction process

Fig. 1 shows that the more threads leads to less computing time to parallel blur degree analysis architecture.

The second experiment is changing the number of thread blocks and keep the number of threads in each thread block unchanged. In the first test of experiment2 and second test of experiment2, the number of thread blocks is set to 1 and 32 respectively.

The experiment shows that the performance of test2 is 9 times better than test 1. So we know that more thread blocks means faster computing speed to parallel blur degree analysis architecture. Since the number of threads is determined by the number of threads in each thread block and the number of thread blocks, we can increase the computing speed of parallel blur degree analysis architecture according to increasing the number of threads.

If the number of threads increases, the number of threads data which need to be reduced will rise. In order to prevent thread data reduction from taking too much computing time, we need a better data reduction algorithm.

III. BINARY REDUCTION ALGORITHM

This chapter provides a binary reduction algorithm. It launches a large number of threads to reduce data and each thread reduces two data. Section A introduces some basic reduction algorithms and section B gives the binary reduction algorithm based on basic reduction algorithms.

A. Basic Reduction Algorithm

Serial reduction algorithm launches a thread to sum NUM integral numbers which store in array *data* and save the result into area which pointer *result* points at.

Algorithm 1 Serial reduction algorithm

```

1: for ( $i = 0; i < NUM; i++$ ) do
2:    $*result = *result + data[i]$ ;
3: end for

```

Group reduction algorithm launches some threads (The number of threads is determined by $NUM/NUM2+1$). Each thread sum $NUM2$ integral numbers which store in array *data* and save the result into array *data_temp*. When all the threads are finished, the first thread sums all the elements in the array *data_temp* and save the result into area which pointer *result* points at.

Algorithm 2 Group reduction algorithm

```

1: int  $a = blockDim.x$ ; //a stores the number of threads
2: int  $b = threadIdx.x$ ; //b stores thread ID
3: if ( $(b + 1) == a$ ) then
4:   for ( $i = 0; i < NUM - b * NUM2; i++$ ) do
5:      $data\_temp[b] += data[b * NUM2 + i]$ ;
6:   end for
7: else
8:   for ( $i = 0; i < NUM2; i++$ ) do
9:      $data\_temp[b] += data[b * NUM2 + i]$ ;
10:  end for
11: end if
12: __syncthreads();
13: if ( $b == 0$ ) then
14:   for ( $i = 0; i < NUM/NUM2 + 1; i++$ ) do
15:      $*result = *result + data\_temp[i]$ ;
16:   end for
17: end if

```

Group reduction algorithm with shared memory is similar to ordinary group reduction algorithm. The difference between two algorithms is algorithm 3 stores addition result of each thread in the global memory and algorithm 4 stores addition result of each thread in the shared memory. Using data in the shared memory is faster than using data in the global memory but the size of shared memory is limited by the hardware.

Algorithm 3 Group reduction algorithm with shared memory

```

1: int  $a = blockDim.x$ ; //a stores the number of threads
2: int  $b = threadIdx.x$ ; //b stores thread ID
3: __shared__ int  $temp[NUM/NUM2 + 1]$ ;
4:  $temp[b] = 0$ ;
5: if ( $(b + 1) == a$ ) then
6:   for ( $i = 0; i < NUM - b * NUM2; i++$ ) do
7:      $temp[b] += data[b * NUM2 + i]$ ;
8:   end for
9: else
10:  for ( $i = 0; i < NUM2; i++$ ) do
11:     $temp[b] += data[b * NUM2 + i]$ ;
12:  end for
13: end if
14: __syncthreads();
15: if ( $b == 0$ ) then
16:   for ( $i = 0; i < NUM/NUM2 + 1; i++$ ) do
17:      $*result = *result + temp[i]$ ;
18:   end for
19: end if

```

B. Binary Reduction Algorithm

Binary reduction algorithm includes GPU part and CPU part. The GPU part is used to reduce data. The CPU part is used to call GPU part and transfer data between host memory

and graphics memory. A simple example of binary reduction algorithm is shown in Fig. 2.

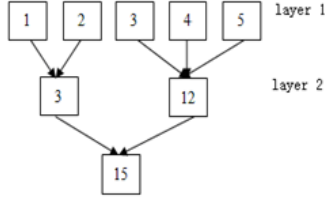


Fig. 2. binary reduction process

Binary reduction algorithm(GPU part) reduces data layer by layer. In the reduction process, one thread reduces two data. If the number of data in the current layer is odd number, the last thread needs to reduce last three data.

Algorithm 4 Binary reduction algorithm-GPU part

```

1: int i = blockIdx.x * blockDim.x + threadIdx.x;
2: //i stores the current thread ID
3: int j = number1;
4: //j stores the number of layers
5: int k = number2;
6: //k stores the number of data in the current layer
7: if (i >= K/2) then
8:   return;
9: end if
10: //array data stores the data which need to be reduced
11: data[(int)(i * pow(2.0, j))] += data[(int)(i * pow(2.0, j) +
   pow(2.0, j - 1))];
12: if (k % 2 != 0 && (i + 1) == (k / 2)) then
13:   data[(int)(i * pow(2.0, j))] += data[(int)(i * pow(2.0, j) +
   2 * pow(2.0, j - 1))];
14: end if

```

Binary reduction algorithm(CPU part) determines the number of threads according to the number of data. When the data in one layer is reduced, the algorithm halves the number of data and continues to reduce data in the next layer until all the data are reduced.

Algorithm 5 Binary reduction algorithm-CPU part

```

1: i = 0;
2: while (j / 2 >= 1) do
3:   if (j > 2 * NUM3) then
4:     if (j / 2 % NUM3 == 0) then
5:       BinaryReduce <<< j/2/NUM3, NUM3 >>>
       (d_data, i + 1, j);
6:     else
7:       BinaryReduce <<< j/2/NUM3 + 1, NUM3 >>>
       (d_data, i + 1, j);
8:     end if
9:   else
10:    BinaryReduce <<< 1, j/2 >>> (d_data, i + 1, j);
11:  end if
12:  j = j/2;
13:  i ++;
14: end while

```

IV. EXPERIMENTS

We conduct two experiments for binary reduction algorithm. The first experiment is using four reduction algorithms to reduce less than 2,000,00 data and compare their computing time. The second experiment is using group reduction algorithm and binary reduction algorithm to reduce more than 2,000,00 data and compare their computing time. All the experiments are performed on AMD Phenom II X6 1055T, 4096M, NVIDIA GTX295/Nvidia GTX480, Windows XP, Visual Studio 2005, CUDA ToolKit 2.3 and CUDA SDK2.3.

A. The Comparison of Four Algorithms

In order to know advantages and disadvantages of four reduction algorithms, we use these algorithms to reduce random integral numbers respectively and compare their computing time. In these algorithms, algorithm 1 is serial algorithm and the others are parallel algorithms. The comparison result of four algorithms is shown in Fig. 3 and Fig. 4.

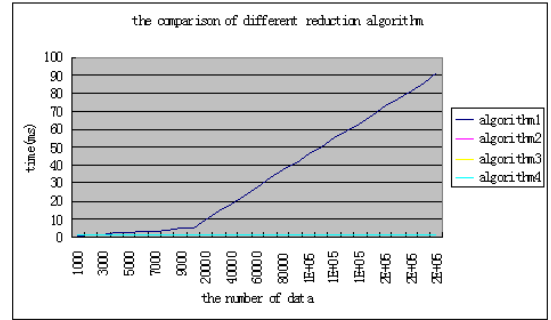


Fig. 3. the comparison of different reduction algorithm

Fig. 3 shows that the computing time of parallel reduction algorithm is far less than the computing time of serial reduction algorithm and their difference will grow up with the rising of data quantity. The computing time in serial algorithm grows linearly and it is relatively stable in parallel algorithm.

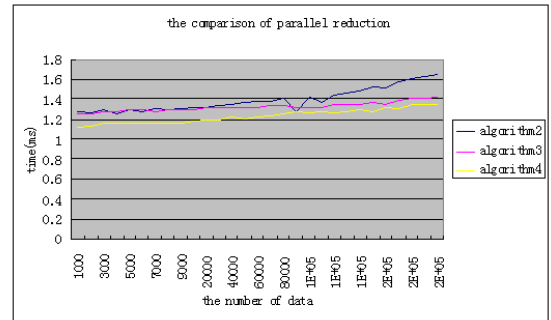


Fig. 4. the comparison of parallel reduction algorithm

In theory, using data in shared memory is faster than using data in global memory. So the computing time of algorithm 3 is less than it of algorithm 2. Since the thread granularity of algorithm 4 is smaller than the thread granularity of algorithm 3, the performance of algorithm 4 is better than it of algorithm 3. Fig. 4 shows that the performance of binary reduction algorithm, the performance of group reduction algorithm with shared memory, the performance of ordinary group reduction algorithm. The experiment result matches theory result.

B. The Comparison of Group Reduction Algorithm and Binary Reduction Algorithm

In algorithm 3, we need to launch more threads to join reduction operation when the number of data grows up. Since the reduction result of each thread stores in the shared memory and the size of shared memory is limited by hardware, algorithm 3 will be failure when the number of data reaches failure threshold. In current experiment environment, failure threshold equals to 2,000,00. In this experiment, we choose two better algorithms i.e. group thread reduction algorithm and binary reduction algorithm from four reduction algorithms according to the previous experiment result to compare their computing time. The number of data in this experiment is more than 2,000,00. The experiment result is shown in Fig. 5.

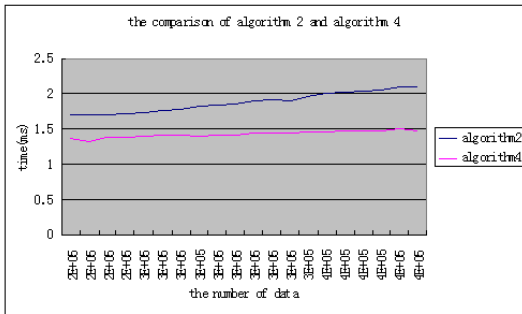


Fig. 5. the comparison of algorithm 2 and algorithm 4

Fig. 5 shows that the performance of binary reduction algorithm is better than group reduction algorithm and their difference will grow up with the rising of data quantity. When the data quantity equals to 4,000,00, the speed of binary reduction algorithm is 42% faster than the speed of group reduction algorithm. Since the computing time of binary reduction algorithm doesn't grow up obviously when the number of data rises, we think this algorithm has good scalability.

V. CONCLUSIONS

This paper provides a binary reduction algorithm based on three simple reduction algorithms according to the CUDA parallel computing model. It launches a large number of threads to reduce data and each thread reduces two data. Experiments show that the performance of it is better than other simple reduction algorithms and it has great scalability.

ACKNOWLEDGMENT

We would like to thank the support of National Nature Science Foundation of China (No.60970012, 61003031), the Innovation Program of Shanghai Science and Technology Commission (No. 09511501000, 09220502800) and Shanghai leading academic discipline project (S30501).

REFERENCES

- [1] Zhang Ying, Li Gen, Pang Zhengbin, Li Yongjin, Zhang Jun, Xia Jun, Yang Xuejun. A Study of the Applicability of Stream Processors to Scientific Computing Applications. *Computer Engineering and Science*. Vol 8:73-81, 2009
- [2] Nvidia Corporation. NVIDIA CUDA compute unified device architecture programming guide 2.0.
- [3] AMD Corporation. ATI Stream Computing Technical Overview.
- [4] Yinghong Sun and Yuanman Tong. CUDA based Fast Implementation of Very Large Matrix Computation. In the 11th International Conference on Parallel and Distributed Computing, Applications and Technologies. 487-491, 2010
- [5] Syed Hasan Adil and Sadaf Qamar. Implementation of Association Rule Mining using CUDA. In 2009 International Conference on Emerging Technologies. 332-336, 2009
- [6] Shuang Wang, Samuel Cheng and Qiang Wu. A Parallel Decoding Algorithm of LDPC Codes using CUDA. In *Asilomar 2008*. 171-175, 2008
- [7] Jing Huang, Sean P. Ponce, Seung In Park, Yong Cao and Francis Quek. GPU-Accelerated Computation for Robust Motion Tracking Using the CUDA Framework. In *VIE08*. 437-442, 2008
- [8] Fubo zuo, Delie Ming, Zhengguang Ao and Wenjuan Zheng. No-Reference Blurred Image Based on Image Structure and the Quality Assessment Method of Its Restoration Image. *GEOMATICS WORLD*. No 1:13-20, 2008
- [9] Zhengfang Zai, Zhiheng Zhou, Huidu Zeng, Shengli Xie. Image blocking effect evaluation standard. *MOBILE COMMUNICATION*. No 1:59-61, 2007
- [10] Xin Huo Ravi, V.T.Wenjing Ma Agrawal, G. Approaches for parallelizing reductions on modern GPUs. In *High Performance Computing (HiPC)*, 2010 International Conference, 2010
- [11] de Veronese, L.P. Krohling, R.A. Differential evolution algorithm on the GPU with C-CUDA. In *Evolutionary Computation (CEC)*, 2010 IEEE Congress, 2010
- [12] Goswami, N. Shankar, R. Joshi, M. Tao Li Intell. Exploring GPGPU workloads: Characterization methodology, analysis and microarchitecture evaluation implications. In *Workload Characterization (IISWC)*, 2010 IEEE International Symposium, 2010
- [13] Yinhuai Sun. The Study of Edge Detection Based on Canny Operator. *Journal of Anshan Normal University*. Vol 12(2):48-50, 2010